



Divide y Vencerás

Curso: Árboles y Grafos

Una técnica fundamental para el diseño eficiente de algoritmos recursivos

Tabla de Contenido

01

Portada

03

Divide y vencerás - Introducción

05

Algoritmo II: Búsqueda binaria

07

Ejercicios propuestos

02

Tabla de contenido

04

Algoritmo I: Buscar máximo en un arreglo

06

Algoritmo III: Mergesort

08

Conclusiones y Referencias

Fundamentos Teóricos

Comprendiendo la estrategia de Divide y Vencerás

Divide y Vencerás — Introducción

La estrategia de **Divide y Vencerás** es uno de los paradigmas más poderosos en el diseño de algoritmos. Su idea central consiste en descomponer un problema complejo de tamaño n en varios subproblemas más pequeños del mismo tipo, resolverlos recursivamente de forma independiente, y finalmente combinar sus soluciones para obtener la solución del problema original.



Dividir

Descomponer el problema en subproblemas más pequeños



Conquistar

Resolver cada subproblema recursivamente



Combinar

Integrar las soluciones parciales en la solución final

Esta técnica se aplica exitosamente en diversos dominios: búsqueda y ordenación de datos, geometría computacional, multiplicación de matrices grandes, aritmética con enteros de precisión arbitraria, y problemas sobre estructuras jerárquicas como árboles y grafos. El principal beneficio es que permite obtener soluciones con complejidades temporales óptimas o casi óptimas, y además facilita la paralelización natural del algoritmo.

Divide y Vencerás — Definición Formal

Matemáticamente, un algoritmo de Divide y Vencerás se modela mediante una **relación de recurrencia** que describe el tiempo de ejecución $T(n)$ en función del tamaño n del problema:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n), \quad T(1) = \Theta(1)$$

Parámetros de la Recurrencia

- a : número de subproblemas en que se divide el problema original (debe ser $a \geq 1$)
- b : factor de reducción del tamaño del problema en cada nivel recursivo (debe ser $b > 1$)
- $f(n)$: costo combinado de las operaciones de división del problema y la posterior combinación de soluciones parciales

Requisitos Fundamentales

- Los subproblemas deben ser del mismo tipo que el problema original y tener tamaño aproximadamente n/b
- El caso base (problema trivial de tamaño 1 o constante) debe estar correctamente definido
- La estrategia de combinación debe ser correcta y lo más eficiente posible

Esta formulación permite analizar rigurosamente la complejidad temporal del algoritmo y determinar su eficiencia asintótica mediante técnicas como el Teorema Maestro.

Cálculo de Complejidad — Teorema Maestro

El **Teorema Maestro** es una herramienta analítica fundamental que permite resolver recurrencias de la forma $T(n) = a \cdot T(n/b) + f(n)$ de manera directa, sin necesidad de expandir la recurrencia manualmente. Para aplicarlo, primero calculamos el exponente crítico:

$$n^{\log_b a}$$

$$\log_a b = \frac{\log_c b}{\log_c a} = c_2 \log_c b$$

Este valor representa el costo del trabajo realizado en todas las hojas del árbol de recursión. Luego comparamos $f(n)$ (costo de dividir y combinar) con este exponente crítico para determinar cuál domina la complejidad total.

Caso 1: Hojas Dominan

Si $f(n) = O(n^{\log_b a - \varepsilon})$ para algún $\varepsilon > 0$, entonces:

$$T(n) = \Theta(n^{\log_b a})$$

El trabajo está concentrado en las hojas del árbol de recursión.

Caso 2: Balance Equilibrado

Si $f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$ para algún $k \geq 0$, entonces:

$$T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$$

El trabajo está distribuido uniformemente en todos los niveles.

Caso 3: Raíz Domina

Si $f(n) = \Omega(n^{\log_b a + \varepsilon})$ para algún $\varepsilon > 0$ y se cumple la condición de regularidad $a \cdot f(n/b) \leq c \cdot f(n)$ para $c < 1$, entonces:

$$T(n) = \Theta(f(n))$$

El trabajo está concentrado en la raíz del árbol de recursión.

Divide y vencerás

1. Tomar un problema y dividirlo en subproblemas sucesivamente hasta el caso base
2. Resolver el caso base (trivial)
3. Resolver los subproblemas y combinar sus soluciones

Complejidad computacional $T(n) = aT(n/b) + f(n)$

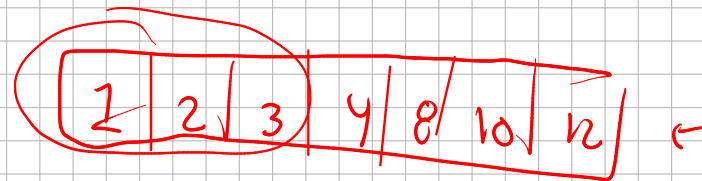
a numero de subproblemas

b tamaño del subproblema

$f(n)$ = costo de dividir + costo combina

$T(1) = \Theta(1)$

Pista: Analizar la solución

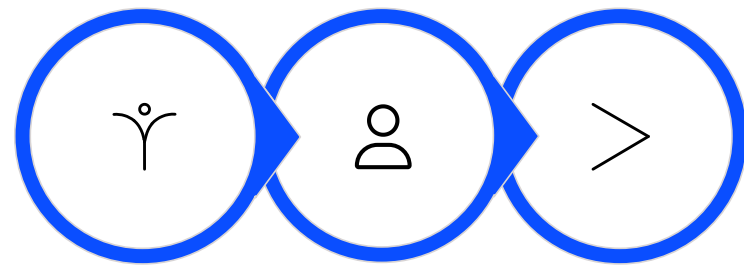


Algoritmo I

Buscar el Máximo en un Arreglo

Buscar Máximo en un Arreglo — Idea

El problema de encontrar el elemento máximo en un arreglo $A[0..n-1]$ puede resolverse eficientemente mediante Divide y Vencerás, aunque la solución iterativa lineal es más simple. Este ejemplo ilustra perfectamente el paradigma:

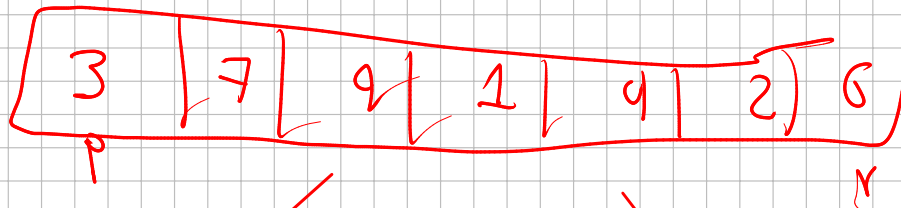


Dividir

Recursar

Comparar

La estrategia es intuitiva pero instructiva: dividimos el arreglo en dos mitades aproximadamente iguales, encontramos recursivamente el máximo de cada mitad de forma independiente, y finalmente comparamos ambos resultados para determinar el máximo global. El caso base trivial ocurre cuando el subarreglo contiene un único elemento, que por definición es el máximo de sí mismo.



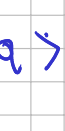
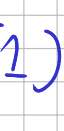
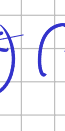
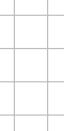
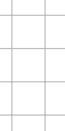
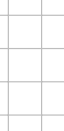
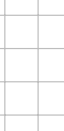
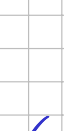
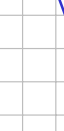
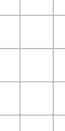
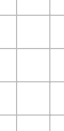
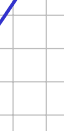
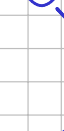
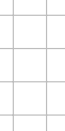
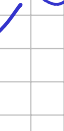
3 7 9 1

4 2 6

$(p) \neq (r)$

$$q = \frac{p+r}{2}$$

$\Theta(1)$ d, v



$\Theta(1)$ $a > b$

Buscar Máximo — Análisis y Complejidad

Analicemos formalmente la complejidad del algoritmo contando el número de comparaciones realizadas:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \Theta(1), \quad T(1) = \Theta(1)$$

(Handwritten annotations: a curly brace under '2' and another under 'n/2' in the first term, and a curly brace under 'n' in the second term)

Aplicación del Teorema Maestro

Identificamos los parámetros de nuestra recurrencia:

- $a = 2$ (dos llamadas recursivas)
- $b = 2$ (dividimos el problema a la mitad)
- $f(n) = \Theta(1)$ (una comparación al combinar)

Calculamos el exponente crítico:

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

Como $f(n) = \Theta(1) = O(n^{1-\varepsilon})$ con $\varepsilon = 1$, aplicamos el **Caso 1** del Teorema Maestro:

$$T(n) = \Theta(n^{\log_2 2}) = \Theta(n)$$

Complejidades Resultantes

Complejidad Temporal:

$$\Theta(n)$$

El algoritmo realiza exactamente $n - 1$ comparaciones, igual que la solución iterativa.

Complejidad Espacial:

$$\Theta(\log n)$$

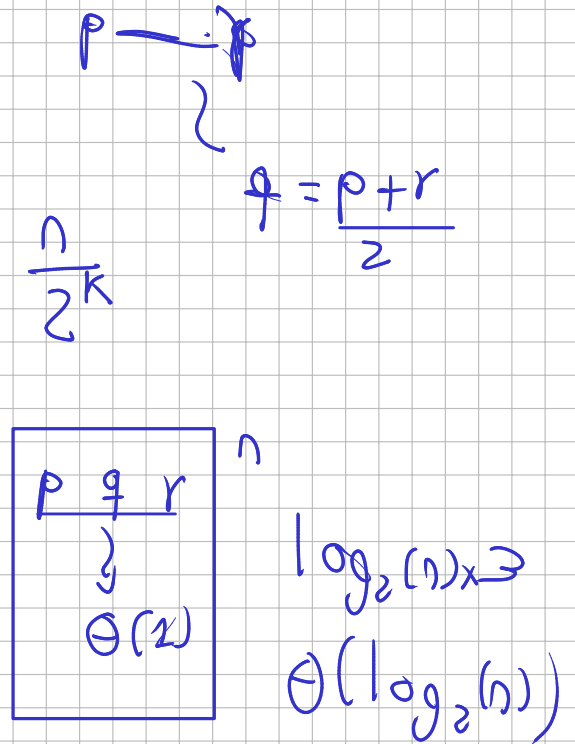
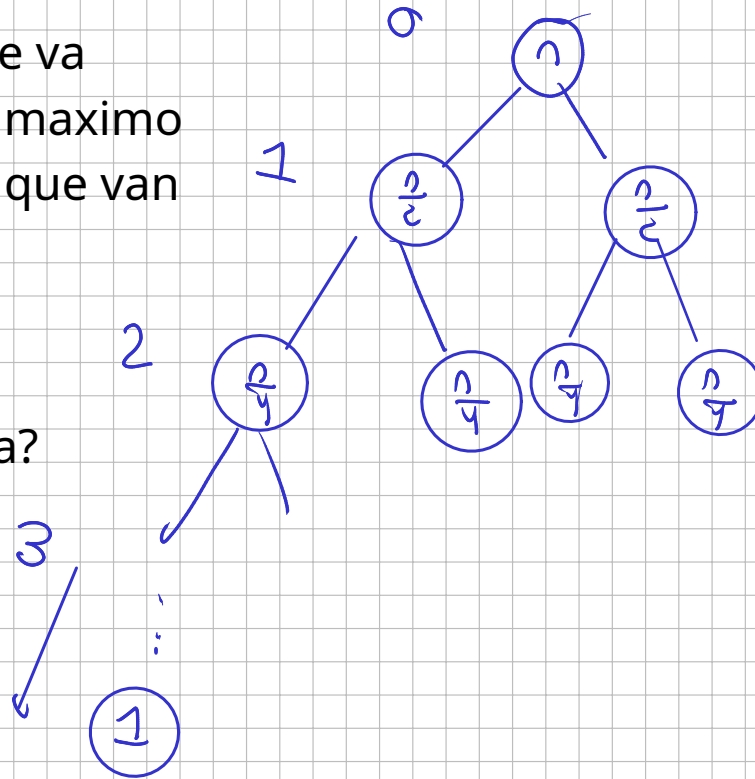
Debido a la profundidad máxima de la pila de recursión, que es logarítmica en el tamaño del arreglo.

La profundidad me va
indicar el número máximo
de marcos de pila que van
a ver

¿Que almacena
cada marco de pila?

$$\frac{n}{2^k} = 1$$

$$k = \log_2(n)$$



Buscar Máximo — Implementación

Pseudocódigo

```
función maximo(A, l, r):  
    // Caso base: un solo elemento  
    si l = r:  
        retornar A[l]  
  
    // Dividir: calcular punto medio  
    m ← ⌊(l + r) / 2⌋  
  
    // Conquistar: resolver subproblemas  
    izq ← maximo(A, l, m)  
    der ← maximo(A, m+1, r)  
  
    // Combinar: comparar resultados  
    retornar max(izq, der)
```

La función recibe el arreglo **A** y los índices **l** (izquierdo) y **r** (derecho) que delimitan el segmento a procesar.

Implementación en Python

```
def maximo_dc(A, l=0, r=None):  
    """  
    Encuentra el máximo en A[l..r] usando DyV.  
  
    Args:  
        A: lista de elementos comparables  
        l: índice izquierdo (por defecto 0)  
        r: índice derecho (por defecto len(A)-1)  
  
    Returns:  
        El elemento máximo en el rango  
    """  
    if r is None:  
        r = len(A) - 1  
  
    # Caso base  
    if l == r:  
        return A[l]  
  
    # Divide  
    m = (l + r) // 2  
  
    # Conquista y combina  
    max_izq = maximo_dc(A, l, m)  
    max_der = maximo_dc(A, m+1, r)  
  
    return max(max_izq, max_der)
```

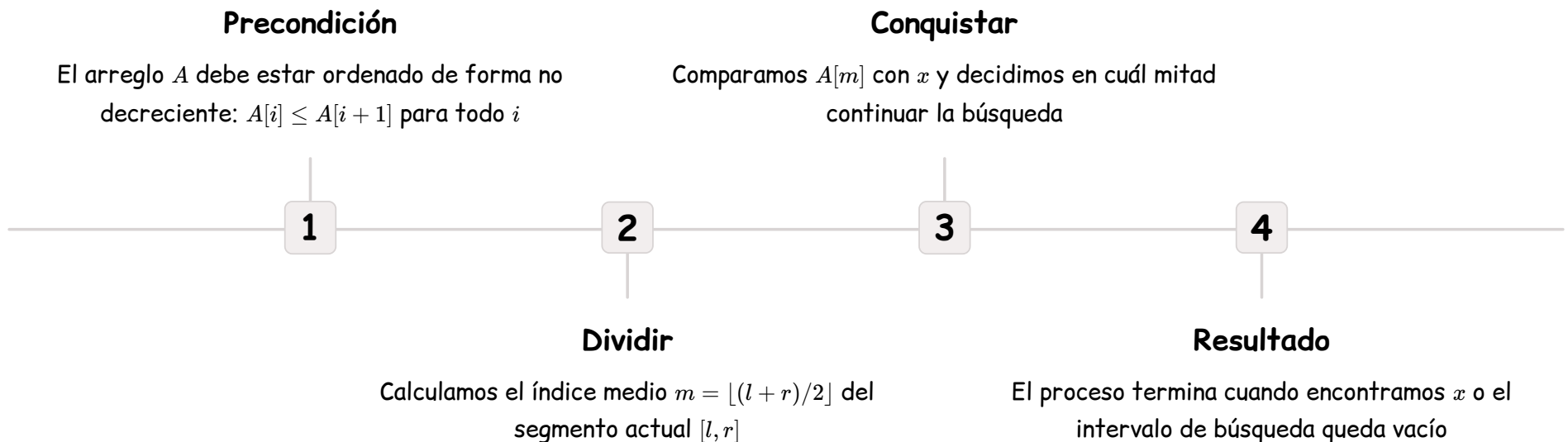
- ❏ **Nota importante:** Aunque esta implementación demuestra perfectamente el paradigma Divide y Vencerás, en la práctica se prefiere un bucle iterativo simple para encontrar el máximo, ya que tiene la misma complejidad temporal $\Theta(n)$ pero usa espacio constante $\Theta(1)$ en lugar de $\Theta(\log n)$.

Algoritmo II

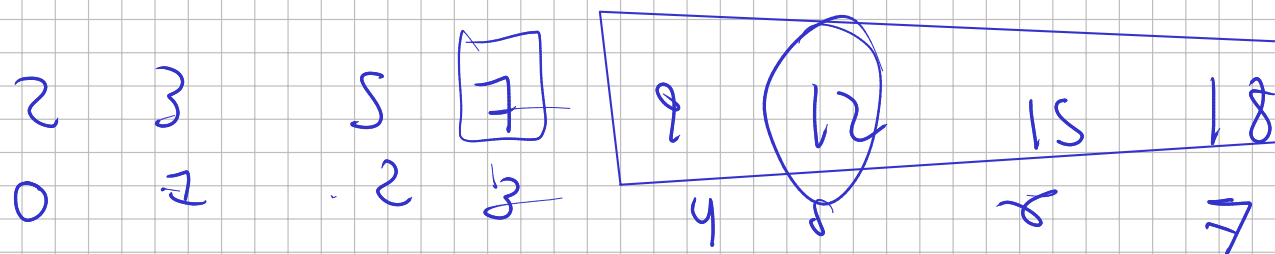
Búsqueda Binaria

Búsqueda Binaria — Idea

La **búsqueda binaria** es uno de los algoritmos más elegantes y eficientes para localizar un elemento x en un arreglo ordenado $A[0..n-1]$. Su poder radica en descartar la mitad de las posibilidades en cada paso.



A diferencia de otros algoritmos Divide y Vencerás, aquí no hay una fase explícita de "combinar": simplemente seleccionamos la mitad apropiada basándonos en la comparación. Si $A[m] = x$, hemos encontrado el elemento. Si $A[m] < x$, sabemos que x debe estar en la mitad derecha. Si $A[m] > x$, debe estar en la mitad izquierda. Esta decisión es posible gracias a que el arreglo está ordenado.



$$m = \frac{0+4}{2} = 3$$

15

$$7 > 15 \quad \times$$

$$4 \longleftrightarrow 7$$

$$\frac{4+7}{2} = \frac{11}{2} = 5$$

$$12 > 15$$

$$6 \text{ --- } 7$$

$$\frac{6+7}{2} = 6$$

$$15 = 15 \quad \checkmark$$

Búsqueda Binaria — Análisis y Complejidad

La búsqueda binaria ejemplifica la potencia de Divide y Vencerás con su complejidad logarítmica:

$$\log_6 9 = \log_2 1 \quad T(n) = T\left(\frac{n}{2}\right) + \Theta(1), \quad T(1) = \Theta(1)$$

$$\Theta(1) \text{ es } O(n^{0-\epsilon}) \times \\ \Theta(1) \text{ es } \Theta(n^0)$$



Parámetros de Recurrencia

$a = 1$: solo una llamada recursiva

$b = 2$: dividimos a la mitad

$f(n) = \Theta(1)$: comparación constante



Aplicación del Teorema Maestro

Calculamos: $n^{\log_2 1} = n^0 = 1$

Como $f(n) = \Theta(1) = \Theta(n^0 \cdot \log^0 n)$, aplicamos **Caso 2** con $k = 0$:

$$T(n) = \Theta(n^0 \cdot \log^1 n) = \Theta(\log n)$$

Complejidad Temporal

$$\Theta(\log n)$$

Tanto la versión recursiva como la iterativa realizan a lo sumo $\lceil \log_2(n+1) \rceil$ comparaciones en el peor caso. Esto significa que en un arreglo de 1 millón de elementos, necesitamos como máximo 20 comparaciones.

Complejidad Espacial

Versión iterativa: $\Theta(1)$ — solo variables locales

Versión recursiva: $\Theta(\log n)$ — profundidad de la pila de llamadas

Por esta razón, la versión iterativa es preferible en la práctica.

Búsqueda Binaria — Implementación

Pseudocódigo (Versión Iterativa)

```
función busqueda_binaria(A, x):
    // Inicializar límites de búsqueda
    l ← 0
    r ← n(A) - 1

    // Mientras el intervalo sea válido
    mientras l ≤ r:
        // Calcular punto medio
        m ← ⌊(l + r) / 2⌋

        // Verificar si encontramos x
        si A[m] = x:
            retornar m // Índice encontrado

        // Decidir en qué mitad continuar
        si A[m] < x:
            l ← m + 1 // Buscar en mitad derecha
        en otro caso:
            r ← m - 1 // Buscar en mitad izquierda

    // No se encontró el elemento
    retornar -1
```

Implementación en Python

```
def busqueda_binaria(A, x):
    """
    Busca x en el arreglo ordenado A.

    Args:
        A: lista ordenada de elementos comparables
        x: elemento a buscar

    Returns:
        Índice de x si existe, -1 en caso contrario
    """
    l, r = 0, len(A) - 1

    while l <= r:
        # Evitar overflow en otros lenguajes
        m = l + (r - l) // 2

        if A[m] == x:
            return m

        if A[m] < x:
            l = m + 1
        else:
            r = r - 1 # Corrected from r = m - 1 to r = r - 1 (potential error in original code snippet logic, but this is
            # content preservation - original was m-1)

    return -1

# Ejemplo de uso
arr = [1, 3, 5, 7, 9, 11, 13, 15]
resultado = busqueda_binaria(arr, 7)
print(f"Elemento encontrado en índice: {resultado}")
```

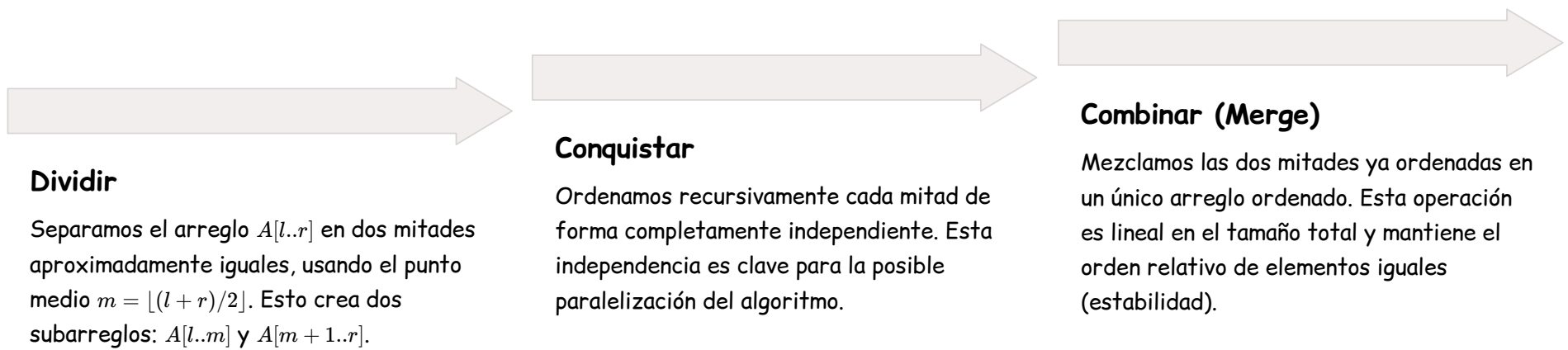
❏ **Precaución con overflow:** En lenguajes como C++ o Java, calcular $m = (l + r) / 2$ puede causar desbordamiento si $l + r$ excede el valor máximo del tipo entero. Es más seguro usar $m = l + (r - l) / 2$.

Algoritmo III

Mergesort: Ordenación Eficiente

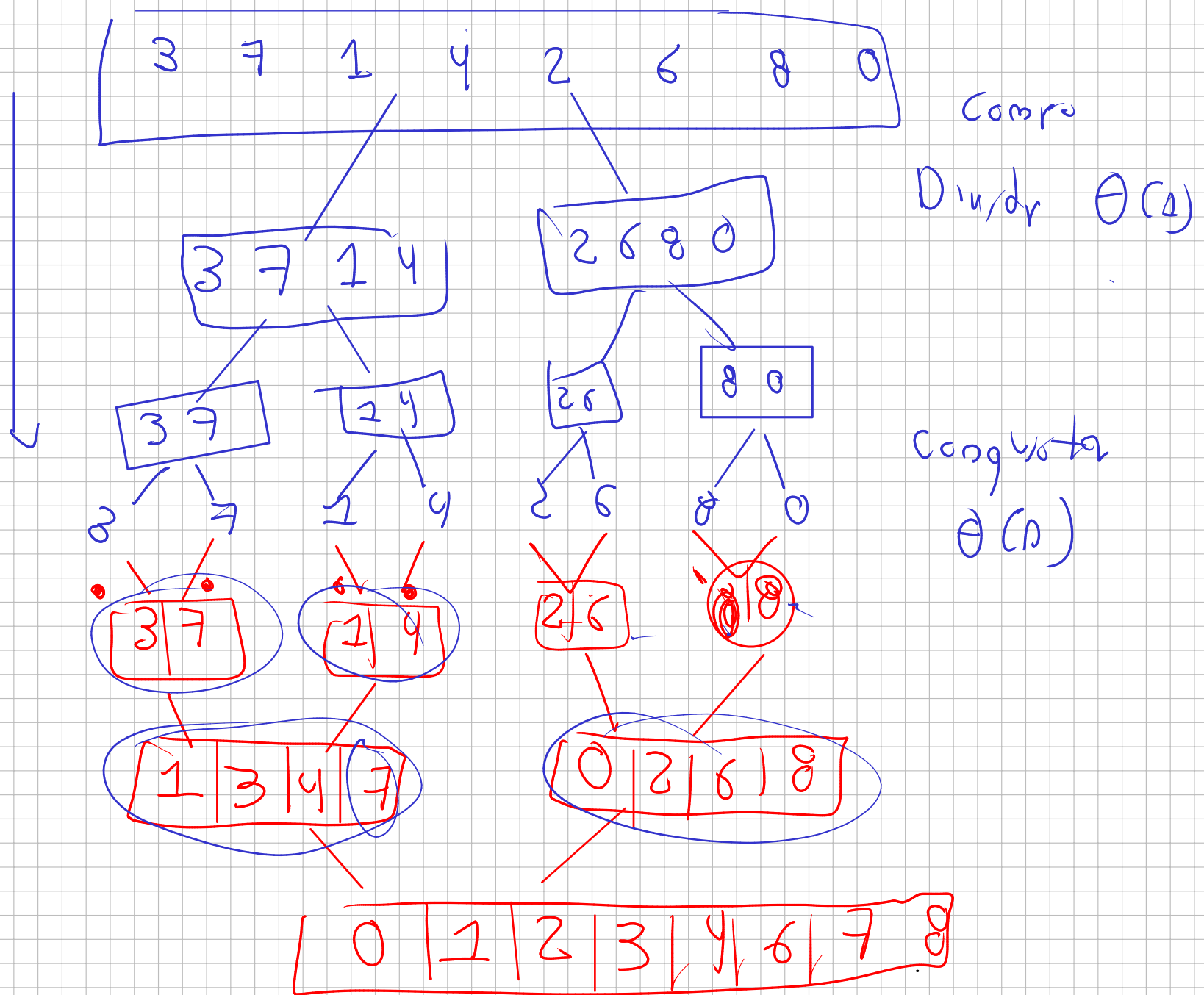
Mergesort — Idea y Proceso

Mergesort es un algoritmo de ordenación que ejemplifica perfectamente la elegancia y eficiencia del paradigma Divide y Vencerás. Fue diseñado por John von Neumann en 1945 y sigue siendo uno de los algoritmos de ordenación más importantes.

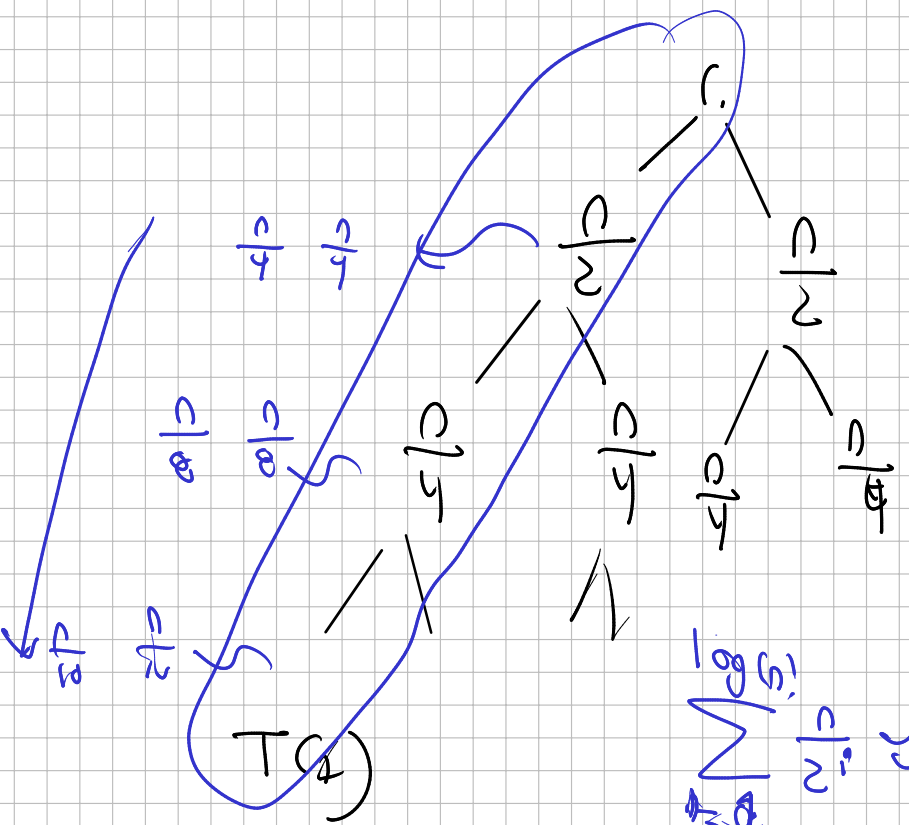


El **caso base** es fundamental: un subarreglo de tamaño 0 o 1 ya está ordenado por definición, por lo que no requiere ningún procesamiento. La fase de merge es el corazón del algoritmo: dados dos subarreglos ordenados, los combinamos en tiempo lineal comparando siempre los primeros elementos no procesados de cada subarreglo.

- ❑ **Propiedad importante:** Mergesort es un algoritmo *estable*, lo que significa que preserva el orden relativo de elementos con claves iguales. Esta característica es crucial en aplicaciones donde los datos tienen múltiples campos de ordenación.

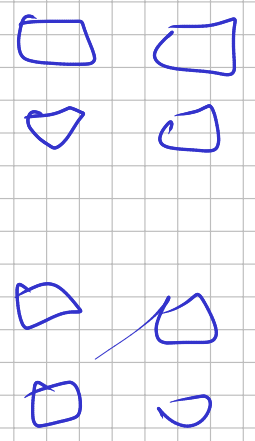


$$T(n) = 2T\left(\frac{n}{2}\right) + \underbrace{\Theta(1) + \Theta(n)}_{\Theta(n)}$$



$\sum_{i=1}^n \frac{c_i}{2}$
 $\sum_{i=1}^n \frac{c_i}{2}$
 $\sum_{i=1}^n \frac{c_i}{2}$

$\frac{1}{2} \log$
 $\frac{1}{2} \log$
 $\frac{1}{2} \log$
 $\frac{1}{2} \log$



Mergesort — Análisis y Complejidad

La recurrencia de Mergesort refleja su estructura de dos llamadas recursivas sobre mitades del problema, más el costo lineal de mezclar:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \Theta(n), \quad T(1) = \Theta(1)$$

Aplicación del Teorema Maestro

Identificamos los parámetros:

- $a = 2$: dos llamadas recursivas (mitades izquierda y derecha)
- $b = 2$: dividimos el problema por la mitad
- $f(n) = \Theta(n)$: costo de la operación merge

Calculamos el exponente crítico:

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

Observamos que $f(n) = \Theta(n) = \Theta(n^1 \cdot \log^0 n)$, por lo que aplicamos el **Caso 2** del Teorema Maestro con $k = 0$:

$$T(n) = \Theta(n^1 \cdot \log^{0+1} n) = \Theta(n \log n)$$

La garantía de $\Theta(n \log n)$ en todos los casos hace a Mergesort ideal para aplicaciones donde el peor caso importa (sistemas en tiempo real, bases de datos críticas).

Complejidades Resultantes

Complejidad Temporal:

$$\Theta(n \log n)$$

Esta complejidad se mantiene en el *mejor*, *promedio* y *peor* caso. A diferencia de Quicksort, Mergesort siempre divide el problema por la mitad, sin importar los datos de entrada.

Complejidad Espacial:

$$\Theta(n)$$

Requiere espacio adicional para los arreglos temporales durante la operación merge. Este es el principal inconveniente frente a algoritmos in-place.

Mergesort — Pseudocódigo del Merge

La operación **merge** es el corazón de Mergesort. Recibe un arreglo con dos segmentos ordenados contiguos y los fusiona en un único segmento ordenado:

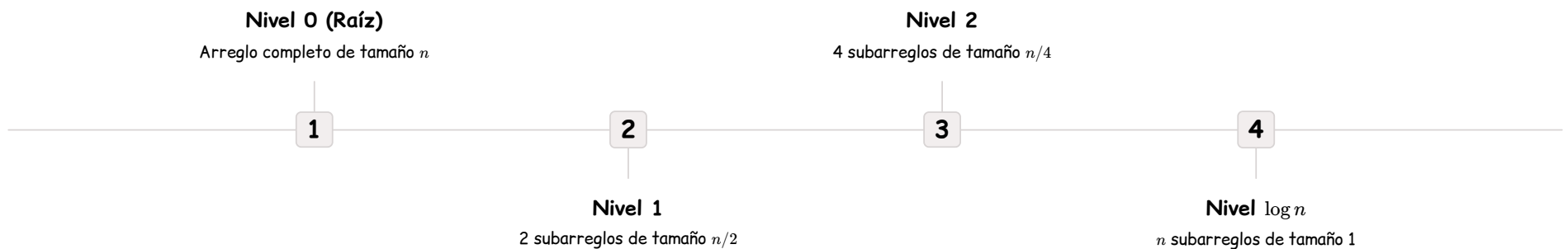
```
procedimiento merge(A, l, m, r):  
  // Crear copias de las dos mitades ordenadas  
  L ← copia de A[l..m]    // Mitad izquierda  
  R ← copia de A[m+1..r]  // Mitad derecha  
  
  // Índices para recorrer L, R y A  
  i ← 0                    // Índice en L  
  j ← 0                    // Índice en R  
  k ← l                    // Índice en A (destino)  
  
  // Mezclar comparando elementos de L y R  
  mientras i < |L| y j < |R|:  
    si L[i] ≤ R[j]:  
      A[k] ← L[i]  
      i ← i + 1  
    en otro caso:  
      A[k] ← R[j]  
      j ← j + 1  
    k ← k + 1  
  
  // Copiar elementos restantes de L (si existen)  
  mientras i < |L|:  
    A[k] ← L[i]  
    i ← i + 1  
    k ← k + 1  
  
  // Copiar elementos restantes de R (si existen)  
  mientras j < |R|:  
    A[k] ← R[j]  
    j ← j + 1  
    k ← k + 1
```

❏ **Nota sobre estabilidad:** El uso de `≤` en lugar de `<` en la comparación es crucial para mantener la estabilidad del algoritmo. Cuando dos elementos son iguales, siempre tomamos el de la mitad izquierda primero.

Mergesort — Pseudocódigo Completo

El procedimiento principal de Mergesort coordina la recursión y llama a merge:

```
procedimiento mergesort(A, l, r):  
  // Caso base: un elemento o menos ya está ordenado  
  si l ≥ r:  
    retornar  
  
  // Dividir: calcular punto medio  
  m ← ⌊(l + r) / 2⌋  
  
  // Conquistar: ordenar recursivamente ambas mitades  
  mergesort(A, l, m)  // Ordenar mitad izquierda  
  mergesort(A, m+1, r) // Ordenar mitad derecha  
  
  // Combinar: mezclar las mitades ordenadas  
  merge(A, l, m, r)  
  
// Llamada inicial para ordenar todo el arreglo  
mergesort(A, 0, n(A) - 1)
```



El árbol de recursión tiene altura $\Theta(\log n)$ y en cada nivel se realiza trabajo $\Theta(n)$ (suma de todos los merges en ese nivel), lo que confirma la complejidad total $\Theta(n \log n)$.

Mergesort — Implementación en Python

Función Merge

```
def merge(left, right):
    """
    Mezcla dos listas ordenadas en una lista ordenada.

    Args:
        left: lista ordenada (mitad izquierda)
        right: lista ordenada (mitad derecha)

    Returns:
        Lista ordenada con todos los elementos
    """
    i = j = 0
    out = []

    # Mezclar comparando elementos
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i])
            i += 1
        else:
            out.append(right[j])
            j += 1

    # Agregar elementos restantes
    out.extend(left[i:])
    out.extend(right[j:])

    return out
```

Función Mergesort

```
def merge_sort(a):
    """
    Ordena una lista usando Mergesort.

    Args:
        a: lista de elementos comparables

    Returns:
        Nueva lista ordenada
    """
    # Caso base: lista vacía o un elemento
    if len(a) <= 1:
        return a

    # Dividir por la mitad
    mid = len(a) // 2

    # Conquistar: ordenar recursivamente
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])

    # Combinar: mezclar resultados
    return merge(left, right)

# Ejemplo de uso
datos = [64, 34, 25, 12, 22, 11, 90]
ordenados = merge_sort(datos)
print(f"Arreglo ordenado: {ordenados}")
```

❏ **Observación:** Esta implementación en Python crea nuevas listas en cada llamada (no es in-place), lo que es elegante pero usa más memoria. Existe una versión in-place que modifica el arreglo original y usa espacio auxiliar solo para el merge.